

MEMOIRE

Mémoire de validation entreprise – 2e année

Module Académique ETNA : PRO-ENT5

Problématique d'Ingénierie :

« Comment faire tenir, dans un même moteur de réservation, des sessions GDS stateful et des offres NDC à durée de vie courte, sans perdre la cohérence des prix ni des dossiers passagers ? »

Candidat :

Jérémy SOLER

Apprenant Expert 2e Année

Spécialité : Architecture Logicielle &
Systèmes Distribués

Poste : Apprenti — Architecte Systèmes
d'Information

Encadrement :

Tuteur Entreprise :

Reouven SAMAMA

Fondateur & CEO — Everbloo

Tuteur Pédagogique :

Comité de Suivi des Mémoires — ETNA

Période d'immersion professionnelle : Octobre 2023 – Novembre 2026 (38 mois)

Année académique 2025 – 2026

Remerciements

Je remercie mon tuteur **Reouven SAMAMA**. Sur Metis, il a validé ou refusé plusieurs choix d'architecture, parfois contre mon avis initial, et j'en ai tiré profit.

Les échanges avec **Jérôme BENBIHI**, **Bernard MOLLE** et **David MARCIANO** m'ont surtout servi quand une règle métier semblait simple côté produit et se révélait impossible tel quel côté API compagnie.

Steeven ALIEL, **Brenda SECK** et **Véronique DANG** ont porté les specs et les recettes. **Jiong Jiong LIN** et l'équipe en Tunisie ont relu beaucoup de merge requests, y compris les plus urgentes un lundi matin.

Merci aussi à l'ETNA et à ma famille.

Résumé

Alternance de 38 mois chez Everbloo (octobre 2023 – novembre 2026) sur la plateforme **Metis**, moteur de réservation aérienne B2B. J’y suis intervenu surtout sur **api-aerial** et **front-reservation**, et ponctuellement sur **front-dashboard** / **api-dashboard**.

Le sujet traité est la coexistence Sabre/Amadeus (GDS) et NDC IATA dans un même parcours de réservation : mapping des offres, sessions OAuth2, création d’ordre (passagers, ancillaries), écarts de prix au paiement, devises ISO 4217, segments passifs. Le reste du document détaille l’implémentation, les mesures et quelques incidents de production.

Mots-clés :

GDS, NDC, Sabre, Amadeus, TypeScript, Node.js, OAuth2, ISO 4217, ADM.

Abstract

I worked for 38 months at SAS Everbloo on **Metis**, a B2B flight booking platform. Most of my code is in **api-aerial** and **front-reservation**. The recurring difficulty was mixing Sabre/Amadeus session-based flows with short-lived NDC offers in one checkout.

I spent time on offer mapping, OAuth2 refresh, NDC order creation (passenger names, lap infants), price changes before ticketing, and multi-currency totals. Chapters below describe the code, the measurements we kept, and a few production incidents.

Keywords:

GDS, NDC, TypeScript, Node.js, OAuth2, ISO 4217, ADM.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
1 Introduction	1
1.1 Contexte	1
1.2 Problématique	1
1.3 Organisation	1
I Contexte et environnement	2
2 Everbloo et mon rôle dans l'équipe	3
2.1 L'entreprise d'accueil	3
2.2 La plateforme Metis	3
2.3 Organisation de l'équipe	3
2.3.1 Mon rôle	3
3 Stack et architecture Metis	5
3.1 Stack et environnement de développement	5
3.2 Découpage des services	5
3.3 Passage à la problématique	6
4 Problématique technique	7
4.1 Écarts GDS / NDC dans le panier	7
4.2 Choix de conception	7
II Réalisations	8
5 GDS et NDC dans le même moteur	9
5.1 Interface de recherche Metis Connect	9
5.2 Normalisation des offres via adaptateurs	9
5.3 Terminal cryptique pour billettistes	12
5.4 Segments passifs Amadeus et Sabre	12
5.5 Synchronisation des PNR	13
6 Réservation, checkout et retarification	14
6.1 Écarts tarifaires (reshopping)	14
6.2 Appairage des passagers avant OrderCreate	15

7 Filtrage multi-compagnies et perfs recherche	18
7.1 Filtres compagnies et préférences	18
7.2 Optimisation des requêtes de recherche	18
8 Multi-devises et calculs tarifaires	19
8.1 Devise du point de vente	19
8.2 Formatage et norme ISO 4217	19
9 Back-Office et paramétrage des agences	20
9.1 Paramétrage des points de vente	20
9.2 Import par lots des profils Sabre	20
10 Tests et mesures en production	21
10.1 Tests unitaires et d'intégration	21
10.2 Mesures en production	21
III Bilan et retours d'expérience	22
11 Limites observées en production	23
11.1 Bilan après 38 mois	23
11.2 Compromis forcés en production	23
11.2.1 Homonymes	23
11.2.2 Segments passifs vs ERP agences	23
11.2.3 CanonicalFlightOffer et les Branded Fares	23
11.2.4 Le 409 ne sauve pas tous les paniers	24
12 Incidents de production et choix d'architecture	25
12.1 OAuth2 : refresh silencieux et file d'attente	25
12.2 Incidents rencontrés en production	26
12.2.1 Écart de centime sur la taxe IZ (taxe Chirac)	26
12.2.2 Horaires Aegean / Turkish décalés de deux heures	27
12.2.3 Deadlock InnoDB sous Sequelize	27
13 Travail en équipe et responsabilités en production	29
13.1 Organisation du développement	29
13.2 Collaboration Paris / Tunisie	29
13.3 Estimations	29
13.4 Incidents côté agences	29
14 Conclusion	30
14.1 Bilan	30
14.2 RNCP 36137	30
14.3 Perspectives	30
Table des sigles et abréviations	31
Glossaire	32
Bibliographie	33

Table des figures

2.1	Logo Everbloo	3
2.2	Logo Metis	3
2.3	Équipe projet	4
3.1	Services Metis et fournisseurs	5
5.1	Interface de recherche de vols Metis Connect	9
5.2	Émulateur de terminal cryptique Sabre	12
6.1	Retarification au checkout	14
6.2	Modale écart de prix	15

Liste des tableaux

3.1 Monorepo Metis — commits	5
--	---

1. Introduction

1.1 Contexte

J'ai effectué mon alternance chez Everbloo d'octobre 2023 à novembre 2026, principalement sur Metis. La plateforme sert des réseaux d'agences — TourCom en particulier — et des comptes entreprises. Pour afficher une offre, elle interroge Sabre, Amadeus et plusieurs connecteurs NDC (Air France-KLM, British Airways, Lufthansa Group, selon les contrats du moment). Dans la pratique, ces sources ne se comportent pas de la même façon : Sabre et Amadeus s'appuient encore beaucoup sur des sessions avec état (EDIFACT, SOAP, parfois du cryptique), alors que NDC expose des API web dont les offres, identifiées par un `OfferID`, expirent au bout de quelques minutes. Les résultats se retrouvent pourtant dans le même panier, et c'est de là que viennent la plupart des problèmes de cohérence de prix et de dossiers passagers que j'ai eu à traiter.

1.2 Problématique

Le tarif vu à la recherche n'est pas toujours celui qui reste disponible à l'émission. Les noms doivent passer les contrôles IATA ; un bébé sans siège doit être rattaché à un adulte ; les devises suivent ISO 4217 ; une écriture passive incorrecte dans le GDS de l'agence peut déclencher un ADM. La question posée en page de garde recouvre ce périmètre. Côté implémentation, l'essentiel tourne autour d'une API Node.js (Sequelize) et d'interfaces Nuxt 3 / Vue 3 en TypeScript.

1.3 Organisation

Le document présente d'abord l'entreprise et la stack, puis les réalisations, les limites observées en production et le lien avec le RNCP. L'ordre n'est pas strictement chronologique.

Première partie I

Contexte et environnement

2. Everbloo et mon rôle dans l'équipe

2.1 L'entreprise d'accueil

Everbloo est l'entreprise où j'ai passé mon alternance (38 mois). Elle est dirigée par **Reouven SAMAMA**, dans le 11^e à Paris. On y développe surtout des outils pour le tourisme et le voyage d'affaires ; TourCom est un client important (réseau d'agences), et il y a aussi « Agence et Vous », branché sur les ERP des agences. Au quotidien, une bonne partie du travail consiste à digérer des protocoles transport et à garder des temps de réponse acceptables quand plusieurs fournisseurs répondent en parallèle — ce qui n'est pas toujours le cas.



FIGURE 2.1 : Logo Everbloo

2.2 La plateforme Metis

C'est sur **Metis** que j'ai passé l'essentiel du temps. Les agents, billettistes et travel managers s'en servent pour chercher et émettre. Le projet associe Everbloo et Metis Digital (**Jérôme BENBIHI**, **Bernard MOLLE**, **David MARCIANO**). Environ une centaine d'agences l'utilisent.



FIGURE 2.2 : Logo Metis

Concrètement, mon travail touchait surtout la recherche multi-fournisseurs, les politiques de voyage, les changements de prix au checkout et le lien avec la comptabilité agence (segments passifs).

2.3 Organisation de l'équipe

Sprints de deux semaines, daily, démos. **Reouven SAMAMA** arbitre avec Metis Digital sur les priorités. Les PO (**Steeven ALIEL**, **Brenda SECK**, **Véronique DANG**) écrivent les tickets Jira et valident. Côté code : moi, **Jiong Jiong LIN**, et des développeurs en Tunisie. Les MR passent par GitLab / GitHub.

2.3.1 Mon rôle

J'ai fait à la fois de l'architecture (adaptateurs Sabre/NDC) et du développement (`api-aerial`, `front-reservation`, un peu `front-dashboard`). Typage TypeScript, tests, et beaucoup de

temps sur les incidents remontés par le support.

Sur la durée : 1 554 commits, +595 310 / -402 680 lignes, 1 869 fichiers touchés.

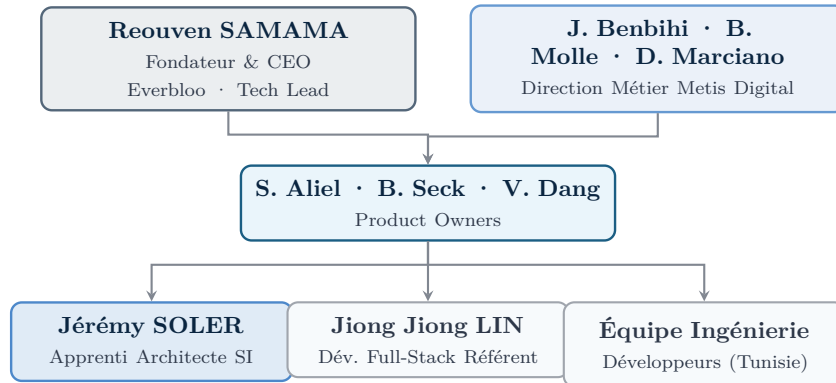


FIGURE 2.3 : Équipe projet

3. Stack et architecture Metis

3.1 Stack et environnement de développement

Le backend tourne sous Node.js / Express. Sequelize gère MySQL et PostgreSQL, avec des migrations versionnées — on a les deux bases parce que le métier aérien et le back-office n’ont pas suivi le même historique. L’auth passe par Keycloak (OpenID Connect, JWT). Les fronts sont en Nuxt 3 / Vue 3 (Composition API), Vuetify 3, Pinia, typés en TypeScript.

En local, Devenv (Nix) et Pixi figent les versions Node et la compilation de modules natifs (bcrypt, libxmljs), ce qui évite surtout les divergences Windows / Linux sur libxmljs. Concurrently lance les services ensemble. Les tests utilisent Mocha / Chai et Playwright.

3.2 Découpage des services

Le moteur de réservation et le back-office sont séparés. Le tableau 3.1 donne la répartition de mes commits.

Composant	Type	Rôle	Commits
api-aerial	Backend Node.js	Agrégation Sabre / Amadeus / NDC, tarifs, PNR.	64.2% (997)
front-reservation	Frontend Nuxt 3	Recherche, panier, retarification, checkout.	33.5% (520)
front-dashboard	Frontend Nuxt 3	POS, profils Sabre, segments passifs.	2.1% (33)
api-dashboard	Backend Node.js	Habilitations, devises agences.	0.3% (4)

TABLE 3.1 : Monorepo Metis — commits

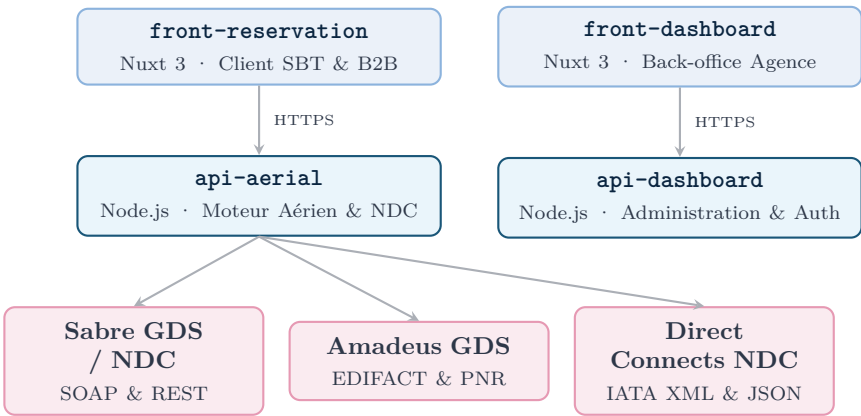


FIGURE 3.1 : Services Metis et fournisseurs

3.3 Passage à la problématique

Sabre répond souvent en 800–1 200 ms ; certaines API NDC mettent 3–6 s, et ce n’est pas stable d’une heure à l’autre. Au checkout, l’offre peut être expirée ou repricee. Les segments passifs et le multi-devises ajoutent un risque ADM qui n’apparaît parfois que des semaines plus tard, quand le transporteur audit le dossier.

4. Problématique technique

4.1 Écarts GDS / NDC dans le panier

En billetterie B2B, une erreur applicative se traduit souvent par une vente bloquée, un voyageur empêché de partir, ou une pénalité financière. Ce n'est pas un détail d'UX.

Sabre, par exemple, conserve un état de session et peut bloquer des places pendant le parcours. Les API NDC raisonnent plutôt en requêtes indépendantes : l'offre vit le temps d'un identifiant temporaire. Metis Connect mélange les deux dans la même interface, ce qui est pratique pour l'agent mais complique tout ce qui touche à la cohérence. Entre la sélection du vol et le paiement, après saisie des passagers, une classe peut se fermer ou des taxes peuvent être recalculées — parfois les deux à la fois. Si l'application se contente de renvoyer une erreur opaque, l'utilisateur doit tout recommencer, y compris les noms IATA déjà validés.

S'ajoutent les contrôles sur les noms, le rattachement des bébés sans siège, les devises, et l'écriture de segments passifs destinés aux logiciels comptables des agences. La question de page de garde recouvre ce périmètre. Certaines limites n'ont apparus qu'après plusieurs mois d'exploitation, pas au moment du design.

4.2 Choix de conception

Pour ne pas dupliquer la logique d'affichage, les réponses transporteurs sont ramenées à un modèle commun avant d'arriver au frontend. Ça a permis de brancher de nouvelles compagnies sans réécrire les écrans du panier. En contrepartie, les offres « branded » ont ensuite forcé un champ fourre-tout côté métadonnées : on n'avait pas prévu de tout typer proprement, et on ne l'a toujours pas fait.

Sur la création d'ordre, un module d'appairage normalise les noms, rattache les nourrissons et retire les bagages orphelins. Le cas de deux homonymes sur le même dossier n'était pas dans les tests initiaux ; le correctif est venu plus tard, après un ticket support.

Lorsque le transporteur signale un conflit tarifaire, une modale affiche l'écart tout en gardant les passagers déjà saisis. Ça aide surtout quand le vol existe encore à un autre prix. Quand le vol n'existe plus, on n'a pas grand-chose à proposer.

Les paramètres d'écriture passive ont été isolés par point de vente, avec une règle empêchant une écriture redondante sur le second GDS. Le reste des frictions avec les ERP agences n'est pas résolu par ce découplage.

Deuxième partie II

Réalisations

5. GDS et NDC dans le même moteur

5.1 Interface de recherche Metis Connect

Metis Connect devait afficher côte à côte des offres Sabre et NDC. Les agents de comptoir utilisent surtout la vue graphique ; les billettistes habitués au GDS conservent un besoin de commandes cryptiques pour certains dossiers. La figure 5.1 montre l'écran principal dans `front-reservation` (Nuxt 3 / Vue 3).

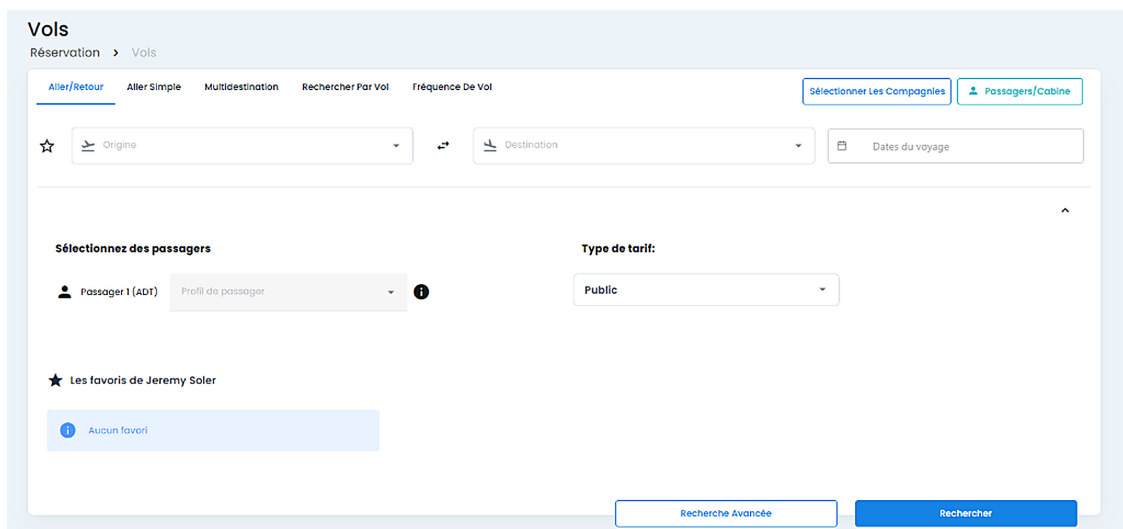


FIGURE 5.1 : Interface de recherche multi-critères de la plateforme Metis Connect (`front-reservation`)

L'écran couvre allers simples, allers-retours et multi-destinations. L'utilisateur renseigne la composition (adultes ADT, enfants CHD, bébés sans siège INF) et choisit tarifs publics ou négociés. Des filtres par compagnies et alliances mettent à jour l'affichage sans recharger la page.

5.2 Normalisation des offres via adaptateurs

Sabre renvoie du SOAP ou du texte brut. Les NDC directs (Air France-KLM, British Airways, Aegean, Turkish) répondent en JSON ou XML, parfois les deux selon l'endpoint. Sans couche commune, le frontend accumulait des branches par transporteur, et chaque nouvelle compagnie recopiait des bugs déjà corrigés ailleurs.

Le listing 5.1 montre `SabreAdapter.ts` : les payloads arrivent dans `CanonicalFlightOffer`. Ensuite, recherche et panier ne voient plus qu'une structure TypeScript. Les limites de ce modèle sont décrites plus loin ; ici, c'est surtout ce qui a permis de livrer sans dupliquer les écrans.

```
1 export interface SabreFlightLeg {
```

```

2   origin: string;
3   destination: string;
4   departureDateTime: string;
5   arrivalDateTime: string;
6   carrierCode: string;
7   flightNumber: string;
8   cabinClassCode: string; // 'Y', 'J', 'F', 'W', etc.
9   bookingClass: string;
10  operatingCarrier?: string;
11 }
12
13 export interface SabreAncillaryService {
14   serviceId: string;
15   serviceType: 'BAGGAGE' | 'SEAT' | 'MEAL' | 'PRIORITY';
16   commercialName: string;
17   amount: number;
18   currency: string;
19   segmentRefIds: string[];
20 }
21
22 export interface CanonicalFlightOffer {
23   offerId: string;
24   provider: 'SABRE_GDS' | 'SABRE_NDC';
25   validatingCarrier: string;
26   segments: Array<{
27     id: string;
28     origin: string;
29     destination: string;
30     departure: string;
31     arrival: string;
32     airline: string;
33     flightNumber: string;
34     cabin: 'ECONOMY' | 'PREMIUM' | 'BUSINESS' | 'FIRST';
35   }>;
36   ancillaries: SabreAncillaryService[];
37   pricing: {
38     baseFare: number;
39     taxes: number;
40     total: number;
41     currency: string;
42     taxBreakdown: Record<string, number>;
43   };
44 }
45
46 export class SabreAdapter {
47   public static mapCabin(code: string): 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |
    'FIRST' {
48     const mapping: Record<string, 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |
    'FIRST'> = {
49       'Y': 'ECONOMY', 'M': 'ECONOMY', 'K': 'ECONOMY', 'L': 'ECONOMY',
50       'W': 'PREMIUM', 'E': 'PREMIUM',
51       'J': 'BUSINESS', 'C': 'BUSINESS', 'D': 'BUSINESS',
52       'F': 'FIRST', 'P': 'FIRST'
53     };

```

```

54     return mapping[code.toUpperCase()] || 'ECONOMY';
55 }
56
57 public static normalizeFlightOffer(raw: any): CanonicalFlightOffer {
58     if (!raw?.id || !Array.isArray(raw?.flights)) {
59         throw new Error("[SabreAdapter] Payload invalide : " + String(raw?.id));
60     }
61
62     const segments = raw.flights.map((leg: SabreFlightLeg, idx: number) => ({
63         id: 'SEG_' + (idx + 1),
64         origin: leg.origin,
65         destination: leg.destination,
66         departure: leg.departureDateTime,
67         arrival: leg.arrivalDateTime,
68         airline: leg.carrierCode,
69         flightNumber: leg.carrierCode + leg.flightNumber,
70         cabin: this.mapCabin(leg.cabinClassCode)
71     }));
72
73     const taxDetails: Record<string, number> = {};
74     if (Array.isArray(raw.taxes)) {
75         raw.taxes.forEach((tax: { code: string; amount: number }) => {
76             taxDetails[tax.code] = (taxDetails[tax.code] || 0) +
77                 Number(tax.amount);
78         });
79     }
80
81     return {
82         offerId: String(raw.id),
83         provider: raw.isNdc ? 'SABRE_NDC' : 'SABRE_GDS',
84         validatingCarrier: raw.validatingCarrier || raw.flights[0].carrierCode,
85         segments,
86         ancillaries: (raw.services || []).map((srv: any) => ({
87             serviceId: srv.id,
88             serviceType: srv.type || 'BAGGAGE',
89             commercialName: srv.name || 'Service Optionnel',
90             amount: Number(srv.price?.total || 0),
91             currency: srv.price?.currency || 'EUR',
92             segmentRefIds: srv.flightRefs || []
93         })),
94         pricing: {
95             baseFare: Number(raw.pricing?.baseFare || 0),
96             taxes: Number(raw.pricing?.totalTax || 0),
97             total: Number(raw.pricing?.totalAmount || 0),
98             currency: raw.pricing?.currency || 'EUR',
99             taxBreakdown: taxDetails
100         }
101     };
102 }

```

Listing 5.1: SabreAdapter.ts

5.3 Terminal cryptique pour billettistes

Pendant longtemps, j'ai sous-estimé l'importance de ce morceau. Pour quelqu'un qui n'a jamais travaillé en agence, une interface graphique semble suffire. En réalité, plusieurs billettistes avec qui j'ai échangé vont plus vite en tapant une commande Sabre qu'en enchaînant des clics, surtout lorsqu'il s'agit de manipuler un PNR hors du parcours standard (profil d'imprimante, classe précise, correction d'un champ).

Plutôt que de les renvoyer vers un émulateur externe, j'ai intégré une modale de terminal dans Metis Connect (figure 5.2). `api-aerial` relaie la saisie via l'API Sabre Command et affiche le texte brut renvoyé. Ce n'est pas le composant le plus spectaculaire du projet, mais c'est l'un de ceux qui ont le plus réduit les frictions au quotidien pour une partie des utilisateurs.

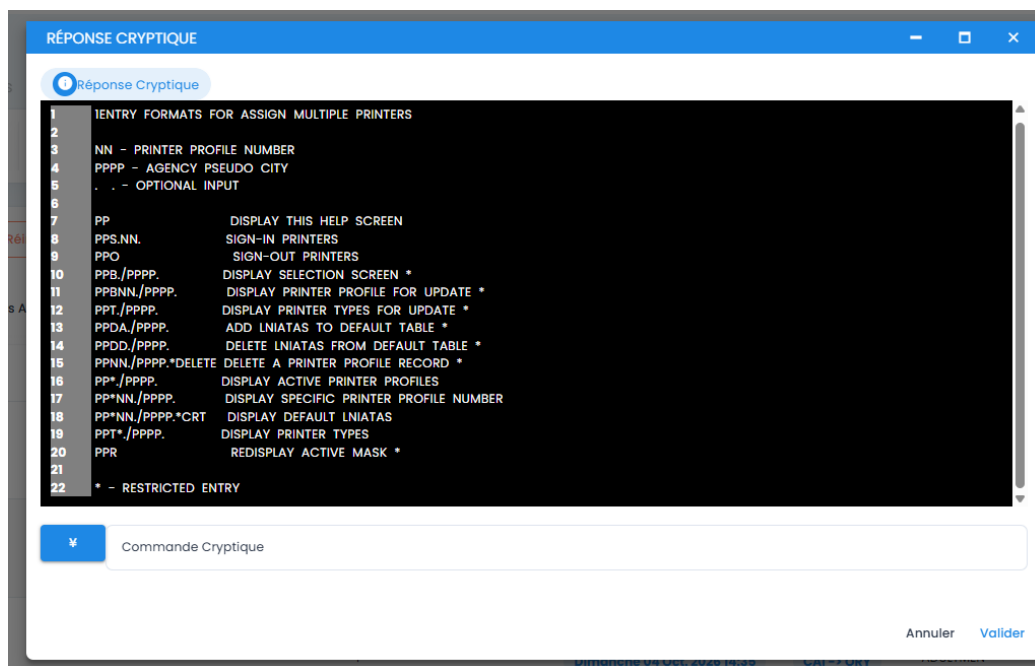


FIGURE 5.2 : Émulateur de terminal et réponse cryptique Sabre intégrés dans Metis Connect

5.4 Segments passifs Amadeus et Sabre

Un billet émis en NDC direct est enregistré chez le transporteur, hors inventaire actif du GDS de l'agence. Pourtant, pour que Gestour, IEnterprise ou l'équivalent génère la facture et les flux de trésorerie, un **segment passif** informatif doit être posé dans le GDS de référence de l'agence.

Une mauvaise coordination produit des doubles écritures passives, ou des écritures incohérentes, ensuite sanctionnées par des avis de débit (*Agency Debit Memos* / ADM). Ce n'est pas théorique : le montant peut atteindre plusieurs milliers d'euros par dossier, et le transporteur n'attend pas que l'on ait fini de déboguer.

La migration `split-passive-segment-options` isole les paramètres d'écriture par point de vente. Avant notification, le contrôleur vérifie le canal d'émission : dossier traité sur Sabre ⇒ écriture passive Amadeus désactivée. Ça coupe le cas le plus fréquent. Ça ne dit rien des ERP qui refusent ensuite la ligne pour une raison de format.

5.5 Synchronisation des PNR

Les PNR évoluent après émission (changement d'horaire, annulation partielle, ajout de SSR). Quand deux agents ouvrent le même dossier, un refresh GDS mal fusionné peut écraser des données locales. La fonction `keepOnlyMatchingTravelers` dans `retrieve.controller.js` ne garde que les voyageurs encore présents côté GDS ; ce n'est pas une synchronisation bidirectionnelle complète, et certains champs annexes restent gérés manuellement.

6. Réservation, checkout et retarification

6.1 Écarts tarifaires (reshopping)

Pendant la saisie des voyageurs, le *Revenue Management* peut fermer un palier ou recalculer taxes et YQ. Le prix vu à la recherche n'est contractuellement fermé qu'à l'émission.

La figure 6.1 décrit le cycle retenu : `repriceOffer`, éventuel HTTP 409, puis confirmation côté front.

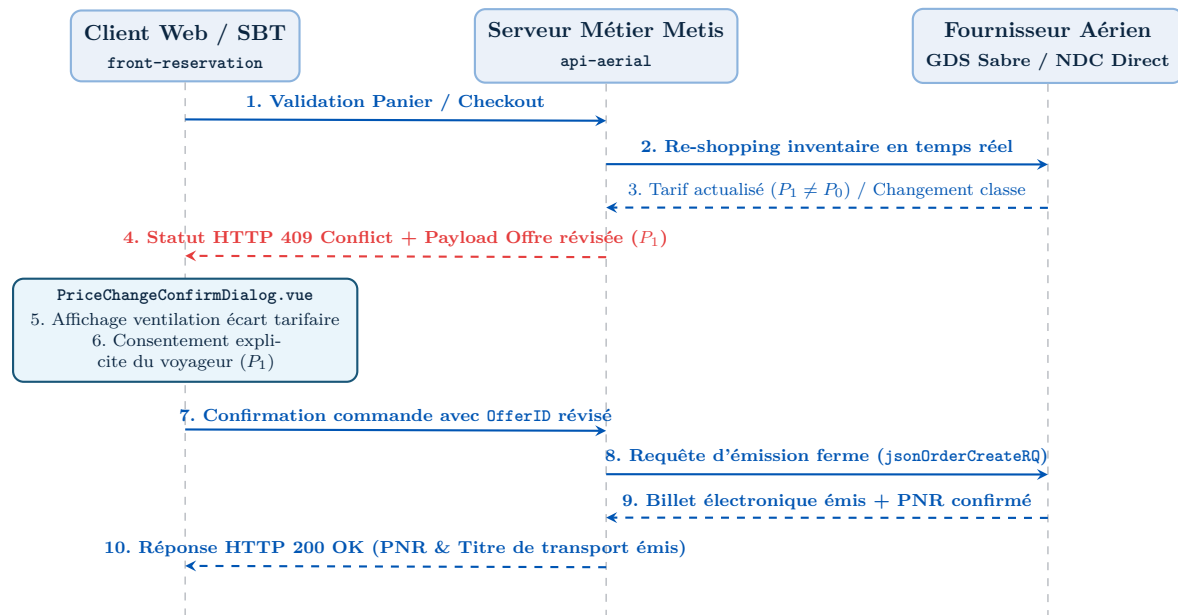


FIGURE 6.1 : Retarification au checkout

À la confirmation d'achat, `api-aerial` envoie un `repriceOffer` au fournisseur. Si le total a bougé ($P_1 \neq P_0$), le backend refuse la création immédiate avec un HTTP 409 (*Conflict*), en joignant la nouvelle grille et un jeton d'offre révisé.

Le frontend intercepte cette réponse métier et ouvre `PriceChangeConfirmDialog.vue` (figure 6.2). La modale détaille l'écart ($\Delta P = P_1 - P_0$) entre base et taxes. Si le voyageur accepte, la commande est rejouée avec le nouvel `OfferID`, en réutilisant les formulaires passagers encore présents dans `useBookingStore`. En cas de refus, le verrou panier tombe et l'utilisateur revient à la liste de résultats.

Changement de tarif

Vol Aller : Paris Charles-de-Gaulle (CDG) → New York (JFK) · Air France AF006

Message transporteur : le tarif de la classe demandée n'est plus disponible au même prix.

Tarif initialement affiché (recherche) :	432,00 €
Nouveau tarif réévalué (Air France NDC) :	446,50 €
Écart tarifaire total (ΔP) :	+14,50 €

Ventilation : +0,00 € sur le tarif de base, +14,50 € sur la surtaxe carburant YQ.

- Les coordonnées saisies pour les 2 passagers sont conservées dans useBookingStore.
- L'offre révisée (OfferID: AF-789-REV) expire dans **04 :38**.

Annuler / Modifier recherche

Confirmer le nouveau tarif (446,50 €)

FIGURE 6.2 : Modale PriceChangeConfirmDialog.vue

6.2 Appairage des passagers avant OrderCreate

Les XML d'erreur NDC (Air France, British Airways) montraient souvent un détail de saisie : « Éléonore », « D'ALMEIDA », bébé sans siège sans rattachement, bagage accroché à un id fantôme après retrait d'un voyageur. Ce n'était pas toujours le même bug ; c'était souvent le même pipeline d'appairage qui laissait passer un état incohérent.

Le listing 6.1 (orderCreation.utils.ts) couvre le nettoyage ASCII, associatedAdultRefId et la purge des services orphelins. Les homonymes, eux, sont restés un trou jusqu'au ticket support décrit plus loin.

```

1 export interface RawTravelerInput {
2   id?: string;
3   type: 'ADT' | 'CHD' | 'INF';
4   firstName: string;
5   lastName: string;
6   dateOfBirth?: string;
7   assignedAdultIndex?: number; // Requis si type === 'INF'
8 }
9
10 export interface NormalizedPassenger {
11   paxRefId: string;
12   ptc: 'ADT' | 'CHD' | 'INF';
13   givenName: string;
14   surname: string;
15   birthDate?: string;
16   associatedAdultRefId?: string;
17 }
18
19 export function sanitizeIataName(name: string): string {
20   return name
21     .normalize('NFD')
22     .replace(/[^\u0300-\u036f]/g, '') // Supprime accents et diacritiques
23     .replace(/[^\u0041-\u005a-\u005b]/g, ' ') // Remplace tirets et apostrophes par
        espaces
24     .replace(/\s+/g, ' ') // Réduit les espaces multiples

```

```

25     .trim()
26     .toUpperCase();
27 }
28
29 export function matchesPaxRefId(
30     travelers: RawTravelerInput[],
31     selectedAncillaries: Array<{ serviceId: string; targetPaxId: string }>
32 ): {
33     passengers: NormalizedPassenger[];
34     validAncillaries: Array<{ serviceId: string; paxRefId: string }>;
35 } {
36     if (!travelers || travelers.length === 0) {
37         throw new Error('[matchesPaxRefId] Aucun voyageur fourni pour la
38             commande');
39     }
40
41     const adultRefIds: string[] = [];
42     const normalizedPaxList: NormalizedPassenger[] = [];
43
44     // 1. Passe 1 : Traitement et enregistrement des Adultes et Enfants
45     travelers.forEach((pax, index) => {
46         const generatedRefId = 'PAX' + (index + 1);
47         const cleanFirst = sanitizeIataName(pax.firstName);
48         const cleanLast = sanitizeIataName(pax.lastName);
49
50         if (pax.type === 'ADT') {
51             adultRefIds.push(generatedRefId);
52         }
53
54         if (pax.type !== 'INF') {
55             normalizedPaxList.push({
56                 paxRefId: generatedRefId,
57                 ptc: pax.type,
58                 givenName: cleanFirst,
59                 surname: cleanLast,
60                 birthDate: pax.dateOfBirth
61             });
62         }
63     });
64
65     // 2. Passe 2 : Traitement des Nourrissons (INF) et rattachement adulte
66     travelers.forEach((pax, index) => {
67         if (pax.type === 'INF') {
68             const generatedRefId = 'PAX' + (index + 1);
69             const adultTargetRef = adultRefIds[pax.assignedAdultIndex ?? 0] ||
70                 adultRefIds[0];
71
72             if (!adultTargetRef) {
73                 throw new Error('[matchesPaxRefId] Nourrisson sans adulte valide : '
74                     + pax.firstName);
75             }
76
77             normalizedPaxList.push({
78                 paxRefId: generatedRefId,

```



```
76     ptc: 'INF',
77     givenName: sanitizeIataName(pax.firstName),
78     surname: sanitizeIataName(pax.lastName),
79     birthDate: pax.dateOfBirth,
80     associatedAdultRefId: adultTargetRef
81   });
82   }
83   });
84
85   // 3. Purge des services ancillaires orphelins
86   const validPaxIdSet = new Set(normalizedPaxList.map(p => p.paxRefId));
87   const validAncillaries = (selectedAncillaries || [])
88     .filter(anc => validPaxIdSet.has(anc.targetPaxId))
89     .map(anc => ({ serviceId: anc.serviceId, paxRefId: anc.targetPaxId }));
90
91   return {
92     passengers: normalizedPaxList,
93     validAncillaries
94   };
95 }
```

Listing 6.1: orderCreation.utils.ts — appairage passagers

7. Filtrage multi-compagnies et perfs recherche

7.1 Filtres compagnies et préférences

En SBT entreprise, les travel managers restreignent souvent les résultats aux alliances contractées (SkyTeam, Star Alliance, Oneworld) ou excluent des low-cost.

`SelectCompanies.vue` propose deux modes : liste blanche (compagnies conventionnées) et liste noire (transporteurs écartés, reste du marché interrogé).

Les préférences vivent dans Pinia pour la session courante et dans le `localStorage` du navigateur. Pas d'aller-retour SQL pour un simple réglage d'affichage.

7.2 Optimisation des requêtes de recherche

Le goulot reste la latence des fournisseurs, pas le SQL local. Dans `api-aerial`, `createShoppingPayload` (`shopping.utils.js`) filtre les compagnies exclues *avant* de construire le XML/JSON, pour ne pas interroger tout le monde puis jeter. Sur les recherches complexes, on gagne souvent quelques centaines de millisecondes ; ce n'est pas spectaculaire, mais ça se sent quand trois connecteurs NDC répondent déjà en plusieurs secondes.

Deux autres réglages, sans rapport l'un avec l'autre : un paramètre 'W' injecté dans les requêtes Sabre pour privilégier les directs (demande fréquente en voyage d'affaires), et des dates normalisées en ISO 8601 via `createDepartureWindow` pour que Sabre et NDC interprètent la même fenêtre horaire. Le second a surtout évité des décalages d'un jour sur les fuseaux.

8. Multi-devises et calculs tarifaires

8.1 Devise du point de vente

Sur le module facturation / conversion, les SQL synchrones qui résolvaient `agencyCurrency` saturaient le pool dès que plusieurs agences lançaient des recherches en parallèle. On avait un paramètre quasi statique qui bloquait tout le pipeline tarifaire, ce qui n'était pas évident tant qu'on regardait uniquement les temps d'appel Sabre.

J'ai déporté la résolution dans un middleware Express asynchrone, avec un cache mémoire de dix minutes. La devise est lue une fois en tête de requête, injectée dans `req.context.posCurrency`, puis réutilisée. Une invalidation Redis serait plus réactive ; pour une donnée que les agences changent rarement, le cache local a tenu. Si quelqu'un change la devise d'un POS en journée, il faut encore attendre l'expiration — on l'a accepté.

8.2 Formatage et norme ISO 4217

Le multi-devises impose ISO 4217. `formatPrice` et `OrderRecap.vue` doivent gérer deux décimales (EUR, USD, GBP), zéro (JPY, KRW) et trois (TND, KWD).

Pour les taxes aéroportuaires, l'arrondi bancaire au pair le plus proche (*banker's rounding*) remplace `Math.round`, qui poussait toujours les demis vers le haut et biaise les totaux sur de gros volumes. On cherche l'égalité entre total facturé et somme des taxes.

9. Back-Office et paramétrage des agences

9.1 Paramétrage des points de vente

front-dashboard (Nuxt 3, Vuetify 3) sert aux administrateurs à configurer les accès techniques de chaque agence.

On y saisit PCC Sabre (*Pseudo City Code*), Office ID Amadeus et numéro IATA. Par réseau : devise par défaut du POS, marge sur les billets, activation des segments passifs selon les besoins comptables.

9.2 Import par lots des profils Sabre

Les imports massifs de profils voyageurs ont saturé les quotas SOAP Sabre tant que les appels partaient tous en parallèle. Les dialogues `DialogUpdateSabre.vue` et `DialogImportProfil.vue` découpent le travail par paquets de 50.

10. Tests et mesures en production

10.1 Tests unitaires et d'intégration

Les calculs de prix et les flux d'émission sont couverts par des tests sur la CI/CD, surtout là où un écart de centime ou un PaxRefID faux se paie en rejet transporteur.

`orderCreation.utils.test.js` (Mocha/Chai) exerce `matchesPaxRefId` sur une quarantaine de cas repris de tickets : bébés multiples, noms à particules, bagages orphelins. `pricingConsistency.utils.test.js` vérifie $\text{Total} = \text{Base} + \sum \text{Taxes}$ sur EUR, USD, TND, JPY (arrondi bancaire). `shopping.utils.test.js` contrôle le filtrage compagnies et le formatage des fenêtres ISO 8601. Je n'ai pas la même densité de tests sur le terminal cryptique ni sur les imports de profils.

10.2 Mesures en production

Les taux d'échec NDC viennent des événements Sentry et des logs Winston sur `api-aerial`, filtrés sur les actions de type création d'ordre. On a exclu les indisponibilités clairement identifiées côté transporteur (timeouts, HTTP 5xx amont). Sur cette définition, avant les correctifs d'appairage, environ 8,4 % des créations d'ordre NDC échouaient pour des raisons applicatives (PaxRefID, services orphelins). Après déploiement, nous sommes restés sous 0,1 %. Ce n'est pas une étude statistique formelle : le mix de compagnies a bougé, et le volume n'est pas constant d'un mois sur l'autre.

Pour les paniers encore associés à une offre après un HTTP 409, nous avons compté, sur la période observée dans les logs front, ceux qui aboutissaient à une émission via la modale — de l'ordre de 92 %. Les cas sans nouvelle offre (vol fermé) n'entrent pas dans ce dénominateur.

La latence de recherche (4,2 s puis 3,0 s en moyenne) a été mesurée lors de campagnes de tests internes, sur 10 000 requêtes multi-fournisseurs, pas sur l'intégralité du trafic production. Sur deux ans d'exploitation suivie, nous n'avons pas relevé d'ADM pour double écriture passive Sabre/Amadeus après le découplage POS.

Troisième partie III

Bilan et retours d'expérience

11. Limites observées en production

11.1 Bilan après 38 mois

Après 38 mois sur Metis, plusieurs briques ont dû être reprises après mise en production. Les points suivants sont ceux qui ont le plus dévié du design initial.

11.2 Compromis forcés en production

11.2.1 Homonymes

Le support a remonté un dossier famille où deux voyageurs portaient le même nom IATA (DUPONT/JEAN). L'appairage s'appuyait alors uniquement sur la clé textuelle normalisée, donc tous les bagages portaient sur le premier passager et la compagnie rejetait à l'émission. Nous avons ajouté l'index de position issu du store (`travelerIndex`) et un avertissement à l'écran. Ça couvre le ticket ; ça reste fragile si quelqu'un réordonne les passagers après coup.

11.2.2 Segments passifs vs ERP agences

Le découplage POS en base a, sur plus de deux ans, éliminé les ADM pour double écriture Sabre/Amadeus — du moins ceux que nous avons pu relier à ce mécanisme. Côté agences, Gestour, IEnterprise et TravelCloud importent selon des conventions très rigides. Certains refusent le PNR passif si la ligne comptable ne colle pas, ou si la devise ne matche pas leur passerelle. Ce n'est pas un bug Metis au sens strict ; c'est un écart de contrat entre ce que le GDS accepte d'écrire et ce que l'ERP veut lire.

Nous avons ajouté une table d'exceptions / règles par agence en PostgreSQL. Les comptables peuvent importer. En pratique, quelqu'un de l'équipe doit encore mettre à jour la table quand un ERP change de format, ce qui n'est ni élégant ni scalable.

11.2.3 `CanonicalFlightOffer` et les *Branded Fares*

L'idée du modèle pivot était de transformer n'importe quel SOAP Sabre ou JSON NDC en une structure commune. Pour environ 90 % des vols simples (aller-retour économique), ça tient. Dès Turkish Airlines ou Lufthansa Group et leurs *Branded Fares*, les offres collent au billet des services non dissociables — bagage 23 kg sur le premier tronçon seulement, sièges limités, conditions d'annulation hybrides — et le typage commence à mentir.

Étendre l'interface TypeScript à chaque particularité alourdissait le modèle plus vite qu'on ne savait le maintenir. Le champ `rawCarrierMetadata: Record<string, any>` a servi d'échappatoire pour afficher ces spécificités sans casser le reste. Je n'ai pas de plan propre pour le retirer.

11.2.4 Le 409 ne sauve pas tous les paniers

L'interception HTTP 409 avec `PriceChangeConfirmDialog.vue` couvre surtout les écarts de taxes modestes ou le basculement sur la sous-classe immédiatement supérieure. Quand le vol est fermé, ou que le saut dépasse le plafond de politique de voyage, l'utilisateur refuse et relance une recherche. Dans ces cas-là, la modale n'est qu'un message d'échec un peu plus lisible.

En test, après un refus, Pinia gardait parfois les identifiants de l'ancienne offre. `useBookingStore().$reset()` à la fermeture de modale a coupé ce cas ; je n'ai pas revérifié tous les chemins de navigation autour.

12. Incidents de production et choix d'architecture

12.1 OAuth2 : refresh silencieux et file d'attente

Entre recherche, choix des vols et saisie des voyageurs, plusieurs minutes s'écoulent. Les access tokens expirent souvent en 15 à 30 minutes. Si l'expiration tombe pendant des appels parallèles (disponibilités, bagages), une implémentation naïve enchaîne des HTTP 401 et déconnecte l'utilisateur en plein checkout.

L'intercepteur Axios (listing 12.1) pose un verrou `isRefreshing` : au premier 401, un seul refresh part ; les autres requêtes rejoignent `failedQueue`. Nouveau token disponible : file vidée, headers mis à jour, requêtes rejouées.

```
1 import axios, { AxiosInstance, AxiosRequestConfig, AxiosError } from 'axios';
2
3 interface QueuedRequest {
4   resolve: (token: string) => void;
5   reject: (error: any) => void;
6 }
7
8 export function setupAuthInterceptor(apiClient: AxiosInstance, authService:
9   any) {
10   let isRefreshing = false;
11   let failedQueue: QueuedRequest[] = [];
12
13   const processQueue = (error: any, token: string | null = null) => {
14     failedQueue.forEach(prom => {
15       if (error) {
16         prom.reject(error);
17       } else if (token) {
18         prom.resolve(token);
19       }
20     });
21     failedQueue = [];
22   };
23
24   apiClient.interceptors.response.use(
25     response => response,
26     async (error: AxiosError) => {
27       const originalRequest = error.config as AxiosRequestConfig & { _retry?:
28         boolean };
29
30       if (error.response?.status === 401 && !originalRequest._retry) {
31         if (isRefreshing) {
32           return new Promise((resolve, reject) => {
33             failedQueue.push({ resolve, reject });
34           })
35             .then(newToken => {
36               if (originalRequest.headers) {
37                 originalRequest.headers['Authorization'] = 'Bearer ' +
```

```
newToken;
36     }
37     return apiClient(originalRequest);
38   })
39   .catch(err => Promise.reject(err));
40 }
41
42 originalRequest._retry = true;
43 isRefreshing = true;
44
45 try {
46   const newToken = await authService.refreshAccessToken();
47   isRefreshing = false;
48   processQueue(null, newToken);
49
50   if (originalRequest.headers) {
51     originalRequest.headers['Authorization'] = 'Bearer ' + newToken;
52   }
53   return apiClient(originalRequest);
54 } catch (refreshError) {
55   isRefreshing = false;
56   processQueue(refreshError, null);
57   authService.handleSessionExpiration();
58   return Promise.reject(refreshError);
59 }
60 }
61
62 return Promise.reject(error);
63 }
64 );
65 }
```

Listing 12.1: Intercepteur Axios OAuth2

12.2 Incidents rencontrés en production

12.2.1 Écart de centime sur la taxe IZ (taxe Chirac)

Le symptôme était un `ERR_PRICE_MISMATCH` sur l'API NDC Air France, réservations Paris – New York facturées en USD. Winston montrait 0,01 USD d'écart entre le total commande et la somme des taxes ventilées (FR, IZ, QW). Ce n'est pas un montant qui saute aux yeux en recette ; c'est le genre d'écart que le transporteur refuse quand même.

On a d'abord regardé du côté de la devise POS injectée dans `req.context.posCurrency`, parce que d'autres tickets de conversion avaient déjà posé problème. La devise collait à celle d'Air France. L'écart venait de l'ordre des opérations : `Math.round` appliqué taxe par taxe après conversion donnait 124,51 USD, alors que la conversion globale du total attendu valait 124,50 USD. La taxe IZ, petite, absorbait souvent le centime de trop dans les logs, d'où le titre du ticket ; le vrai sujet était l'arrondi, pas la taxe Chirac en tant que telle. Nous avons basculé sur l'arrondi bancaire et ajouté une passe finale dans `pricingConsistency.utils.js` où la dernière composante de taxe absorbe le centime ($\text{Total} = \text{Base} + \sum \text{Taxes}$). Ça a calmé les

rejets Air France sur ce motif, sans rien dire des autres compagnies qui ventilent autrement.

Le listing 12.2 reproduit la trace capturée.

```
1 {
2   "timestamp": "2024-04-18T14:22:09.142Z",
3   "level": "warn",
4   "service": "api-aerial",
5   "correlationId": "req-b2b-7f89a1c2",
6   "pos": "NYC_0042",
7   "action": "ORDER_CREATE_REJECTED",
8   "supplier": "AIR_FRANCE_NDC",
9   "error": {
10    "code": "ERR_PRICE_MISMATCH",
11    "httpStatus": 409,
12    "expectedTotal": "124.50 USD",
13    "calculatedTaxSum": "124.51 USD",
14    "breakdown": {
15      "baseFare": "850.00 USD",
16      "taxFR": "45.20 USD",
17      "taxIZ_Solidarity": "18.31 USD",
18      "taxQW_Airport": "61.00 USD"
19    }
20  }
21 }
```

Listing 12.2: Log Winston — rejet NDC écart de taxes

12.2.2 Horaires Aegean / Turkish décalés de deux heures

Les alertes venaient des agences, pas d'un test interne : vols Athènes et Istanbul dont l'heure affichée changeait selon que le poste était à Paris ou à Londres. Les XML renvoyaient des dates ISO sans suffixe de fuseau ("2024-06-15T18:30:00"). `new Date()` côté Node les prenait en UTC ; le navigateur réappliquait le fuseau local. On a fini par parser avec `date-fns-tz` et une table aéroport IATA → fuseau IANA. Le décalage de deux heures n'était pas « le bug Aegean » : c'était le même parsing sur deux compagnies qui omettent l'offset.

12.2.3 Deadlock InnoDB sous Sequelize

Pendant une campagne promo chez une agence partenaire, les réservations simultanées ont commencé à échouer avec :

```
Deadlock found when trying to get lock; try restarting transaction
```

J'ai d'abord augmenté `connectionLimit`, ce qui n'a rien fait : les erreurs restaient sur les écritures de dossier, pas sur les lectures de recherche. Un `retry` avec `setTimeout` a fait baisser un peu le taux d'erreur, assez pour que le support respire, pas assez pour que je considère le sujet clos.

La repro n'est pas venue toute seule. Un script qui lançait plusieurs `OrderCreate` en parallèle sur le même POS a fini par la produire. Les traces InnoDB montraient deux chemins d'écriture qui ne prenaient pas les tables dans le même ordre — une transaction faisait `UPDATE PointDeVente` puis `INSERT Dossier`, l'autre l'inverse. C'est ce que `SHOW ENGINE INNODB STATUS` a renvoyé ce jour-là, après un moment à regarder ailleurs.

Nous avons imposé le même ordre d'update dans Sequelize, passé l'isolation à `READ COMMITTED`, et posé un `SELECT ... FOR UPDATE` trié par clé primaire avant la mise à jour des compteurs. Les retries applicatifs sont restés, parce que je n'étais pas convaincu d'avoir vu tous les chemins concurrents.

13. Travail en équipe et responsabilités en production

13.1 Organisation du développement

Les développements suivaient des sprints de deux semaines, avec daily et recettes PO. Sur les sujets sensibles — tarification, écritures passives, OAuth2 — les merge requests passaient par une revue systématique sur GitLab ou GitHub. Ce n'était pas le cas de tout le reste : j'ai aussi fusionné des correctifs UI sans grande discussion quand le support attendait. Plusieurs refontes (retarification, unification partielle GDS/NDC) ont nécessité une phase de cartographie des flux avant le codage, faute de quoi les correctifs locaux dans les contrôleurs se cumulaient.

13.2 Collaboration Paris / Tunisie

L'équipe de développement est répartie entre Paris et la Tunisie. Les décisions techniques sont documentées (wiki, commentaires de revue), mais le décalage horaire fait que certaines discussions se terminent en commentaires de MR plutôt qu'en visio. Après l'incident des fuseaux, on a ajouté une règle de revue : pas de `new Date()` sur une chaîne ISO sans offset. Ça n'empêche pas d'oublier ailleurs.

13.3 Estimations

Les premiers chiffrages sur les *ancillaries* NDC sous-estimaient la diversité des règles compagnies. Les tests d'émission avec bébé sans siège ont entraîné une réécriture du mapping. Sur SOAP Sabre, le temps réel a surtout été consommé par la gestion des sessions expirées et des réponses non nominales.

13.4 Incidents côté agences

Lors d'une indisponibilité liée à un certificat intermédiaire expiré sur la passerelle Sabre, le support TourCom a signalé un blocage massif en matinée. Les écarts de taxes ou les doubles segments passifs restent par ailleurs une source d'ADM dont le montant peut atteindre plusieurs milliers d'euros par dossier.

14. Conclusion

14.1 Bilan

Les adaptateurs, l'intercepteur OAuth2, l'appairage des passagers et la retarification sont en production sur le parc d'agences que nous suivons. Cela ne clôt pas le sujet : `rawCarrierMetadata` reste un champ fourre-tout, la table d'exceptions ERP doit encore être tenue à jour à la main, et le parcours après un 409 sans nouvelle offre est toujours médiocre. J'ai aussi passé plus de temps que prévu sur trois incidents (taxe IZ, fuseaux Aegean/Turkish, deadlock InnoDB). Depuis, je prépare plus souvent une reproduction avant de modifier un contrôleur, et je pose la question au PO lorsqu'une règle métier est floue.

14.2 RNCP 36137

Pour le titre d'**Expert en Ingénierie Informatique**, ce projet m'a confronté à une architecture multi-services, au full-stack Node.js / Nuxt, aux contraintes ADM, aux tests automatisés et au travail avec une équipe répartie Paris / Tunisie. Je n'ai pas déroulé le référentiel point par point ; ce sont simplement les sujets que j'ai réellement traités.

14.3 Perspectives

Deux sujets que je suivrais en priorité : la veille **IATA ONE Order**, qui pourrait simplifier le modèle de commande, et un traçage **OpenTelemetry** des appels transporteurs, aujourd'hui surtout diagnostiqués via Winston et Sentry.

Table des sigles et abréviations

Sigles métier et protocoles utilisés dans le mémoire. Les termes généraux de génie logiciel (API, JSON, REST, etc.) ne sont pas repris ici.

Sigle / Terme	Définition métier
ADM	<i>Agency Debit Memo</i> — Avis de débit émis par un transporteur à une agence (souvent pour écart tarifaire ou écriture passive incorrecte).
EDIFACT	<i>Electronic Data Interchange for Administration, Commerce and Transport</i> — Standard d'échange historique des GDS.
EMD	<i>Electronic Miscellaneous Document</i> — Titre électronique pour services auxiliaires payants (bagage, siège, etc.).
GDS	<i>Global Distribution System</i> — Réseau de distribution (Sabre, Amadeus).
IATA	<i>International Air Transport Association</i> .
ISO 4217	Codification et précision monétaire des devises.
NDC	<i>New Distribution Capability</i> — Standard IATA de distribution aérienne directe (XML/JSON).
OCN	<i>Order Change Notification</i> — Notification asynchrone de modification de commande ou de vol.
PCC	<i>Pseudo City Code</i> — Identifiant marchand Sabre d'un point de vente.
PNR	<i>Passenger Name Record</i> — Dossier passager (souvent 6 caractères).
POS	<i>Point of Sale</i> — Point de vente (droits, devise, contrats).
SBT	<i>Self-Booking Tool</i> — Outil de réservation pour voyages d'affaires.
SOAP	<i>Simple Object Access Protocol</i> — Enveloppes XML encore utilisées par certains connecteurs GDS.
SSR	<i>Special Service Request</i> — Demande de service spécial passager dans un PNR.

Glossaire

Notions propres à la distribution aérienne et au moteur Metis.

Terme		Définition
Ancillaries		Options vendues avec le billet (bagage soute, siège, repas, embarquement prioritaire).
Branded Fare		Offre tarifaire groupant un tarif de base et des services non dissociables (bagage inclus sur un tronçon, conditions d'annulation, etc.).
Debit (ADM)	Memo	Pénalité financière transporteur → agence pour non-respect des règles tarifaires ou d'écriture.
Office ID		Identifiant marchand Amadeus d'un point de vente.
PaxRefID		Identifiant passager dans un message NDC, utilisé pour lier voyageur, billet et services.
Reshopping / Re-price		Re-tarification avant émission pour valider le prix courant et, le cas échéant, obtenir un nouvel OfferID.
Segment passif		Ligne de vol informative inscrite dans un GDS pour la comptabilité, sans prise d'inventaire.
Taxe IZ		Taxe de solidarité (dite « taxe Chirac ») figurant dans la ventilation des taxes aéroportuaires françaises.

Bibliographie

Standards et spécifications

- [1] **Hardt, D.** (ed.) (2012). *The OAuth 2.0 Authorization Framework*. RFC 6749. IETF. Sections 1.5 (Refresh Token) et 10 (Security Considerations). <https://datatracker.ietf.org/doc/html/rfc6749>
- [2] **Jones, M., Bradley, J., & Sakimura, N.** (2015). *JSON Web Token (JWT)*. RFC 7519. IETF. <https://datatracker.ietf.org/doc/html/rfc7519>
- [3] **International Organization for Standardization.** (2015). *ISO 4217:2015 — Codes for the representation of currencies and funds*. ISO, Geneva. (précision décimale par devise).
- [4] **International Air Transport Association (IATA).** (2021). *NDC Implementation Guide*, version 21.3. Messages AirShoppingRQ/RS, OfferPriceRQ/RS, OrderCreateRQ/RS ; éléments Pax, PaxRefID, OrderItem. IATA Publications, Montreal–Geneva. Guide payant, pas d’URL publique stable.
- [5] **International Air Transport Association (IATA).** (2023). *ONE Order* — architecture et spécifications techniques. IATA Future Airline Distribution, Geneva.
- [6] **Sabre Corporation.** (2024). *Sabre APIs Developer Reference* (SOAP / REST). Sabre Dev Studio, Southlake, Texas. Endpoints Session / Token Create et Command (cryptique).
- [7] **Amadeus IT Group.** (2023). *Amadeus Web Services Integration Guide* — Passive Segments & PNR Management. Madrid.

Documentation d’exécution et retours d’expérience

- [T-1] **Sequelize.** *Transactions* (niveaux d’isolation, verrous). Documentation v6. <https://sequelize.org/docs/v6/other-topics/transactions/> (consulté en 2026).
- [T-2] **Sequelize.** Issue #8294 — *How to correctly retry transactions* (deadlocks MySQL, `retry.match: [/Deadlock/i]`). Issue fermée, fil conservé. <https://github.com/sequelize/sequelize/issues/8294>
- [T-3] **Loon, T.** (2021). *How to Handle Database Deadlock in Sequelize*. JavaScript in Plain English. <https://javascript.plainenglish.io/how-to-handle-database-deadlock-in-sequelize-c> (consulté en 2026 ; copie aussi sur <https://plainenglish.io/javascript/how-to-handle-database-deadlock-in-sequelize-c>)
- [T-4] **Oracle MySQL.** *InnoDB Locking and Transaction Model* — deadlocks et SELECT ... FOR UPDATE. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking.html>
- [T-5] **Axios.** Interceptors — documentation officielle (file d’attente / retry sur 401). <https://axios-http.com/docs/interceptors>

- [T-6] **date-fns. date-fns-tz** — parsing de dates sans offset et association fuseau IANA. <https://github.com/marnusw/date-fns-tz>
- [T-7] **Vue.js / Pinia**. Documentation Composition API et stores typés. <https://pinia.vuejs.org/>; <https://vuejs.org/guide/introduction.html>
- [T-8] **Nuxt**. Documentation Nuxt 3 (installation, routing). <https://nuxt.com/docs/3.x/getting-started/installation> (consulté en 2026).

Ouvrage de référence (patron Adaptateur)

- [B-1] **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns*. Addison-Wesley. Chapitre « Adapter » — utilisé pour cadrer `SabreAdapter.ts` / `CanonicalFlightOffer`.